

# Final Project Report

**Suicide Ideation Detection — Improved Baseline NLP Model**  
**Author: Ritu Patel**

## 1. Introduction

The goal of this project is to automatically detect suicide ideation from social media text, specifically Reddit posts. Early detection of suicidal intent is a critical task for digital mental health support and crisis intervention. Our group implemented several Natural Language Processing (NLP) methods to classify whether a Reddit post indicates suicidal ideation or non-suicidal content.

This is a team project with three distinct model approaches:

**Table: Effort Distribution**

| Team Member     | Method                       |
|-----------------|------------------------------|
| Ritu Patel (Me) | TF-IDF + Logistic Regression |
| Anusha          | LSTM Variant                 |
| Yeji            | BERT / RAG-based System      |
| Fardin          | Streamlit                    |

My primary responsibility was to design, implement, improve, and analyze the baseline classical NLP model. This model serves as a benchmark for comparing against more complex architectures.

This report focuses solely on the system that I developed.

## 2. Dataset Description

The dataset originates from posts collected via the Pushshift API from mental-health-related Reddit communities between 2008 and 2021. It is a subset used for suicide ideation detection research (IEEE: "Suicide Ideation Detection in Social Media Forums").

- Total posts: 232,074

- Two labels:
  - suicide → from r/SuicideWatch
  - non-suicide → from r/teenagers and r/depression
- Features provided: title, post\_text, class

I combined fields to form the final text input:

- `df['text'] = df['title'].fillna('') + ' ' + df['post_text'].fillna('')`
- `df['label'] = df['class']`

The dataset is perfectly balanced:

**Table: Dataset Description**

| Label       | Count   |
|-------------|---------|
| suicide     | 116,037 |
| non-suicide | 116,037 |

This ensures reliable metric interpretation.

### 3. Model Description and Background

I implemented a classical supervised text classification system using Logistic Regression and TF-IDF.

#### Logistic Regression

This is a linear classifier trained to predict probabilities:

$$\hat{y} = \sigma(Wx + b)$$

Where:

- $(x)$  is the TF-IDF feature vector
- $(W)$  learns contribution of each term to suicide risk

Logistic Regression was chosen because:

- It is lightweight and scalable for large text datasets
- The coefficients are interpretable and reveal risk-related vocabulary
- It performs strongly on bag-of-words tasks

## TF-IDF Vectorization

Text is transformed into numerical features using:

```
TfidfVectorizer(  
    max_features=100000,  
    ngram_range=(1, 3),  
    sublinear_tf=True,  
    min_df=3  
)
```

Justification:

- Higher-order n-grams capture phrase-level suicidal intent (e.g., “want to die”)
- Larger vocabulary improves recall of varied language
- Sublinear term frequency reduces exaggeration of frequent words

## 4. Experimental Setup

The dataset was preprocessed through cleaning, lemmatization, and stopword removal, then split into stratified train, validation, and test sets. Text was transformed using TF-IDF with extended n-grams to capture contextual patterns. Logistic Regression with class-balanced weighting served as the baseline classifier. Hyperparameter tuning with GridSearchCV ensured optimal performance. Model evaluation was conducted using accuracy, macro-F1, ROC-AUC, and confusion matrices to assess both overall performance and error distribution.

**Table: Experimental Setup**

| Component | Method |
|-----------|--------|
|-----------|--------|

|                       |                                                            |
|-----------------------|------------------------------------------------------------|
| Train/Val/Test Split  | 80/10/10 stratified                                        |
| Text cleaning         | Lowercasing, URL removal, lemmatization, stopwords removal |
| Class imbalance       | <code>class_weight = 'balanced'</code>                     |
| Hyperparameter tuning | GridSearchCV, scoring = macro F1                           |
| Evaluation Metrics    | Accuracy, precision, recall, macro F1, ROC-AUC             |
| Frameworks used       | Scikit-Learn, NLTK, Pandas, Seaborn, Matplotlib            |

#### Hyperparameters searched:

```
params = {
    "C": [0.5, 1, 5, 10],
    "penalty": ["l2"],
    "solver": ["lbfgs"]
}
```

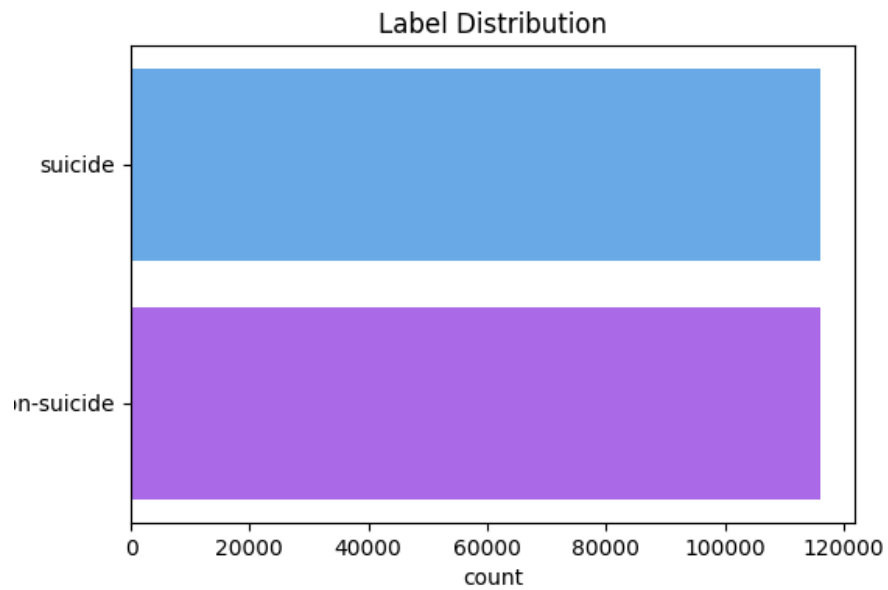
#### Best Parameters:

```
{'C': 10, 'penalty': 'l2', 'solver': 'lbfgs'}
```

## 5. Results and Visual Insights

### 5.1 Label Distribution

The label distribution plot confirms that the dataset is perfectly balanced, with an equal number of suicide and non-suicide posts. This ensures that evaluation metrics such as accuracy and F1-score remain reliable and are not biased toward a majority class.

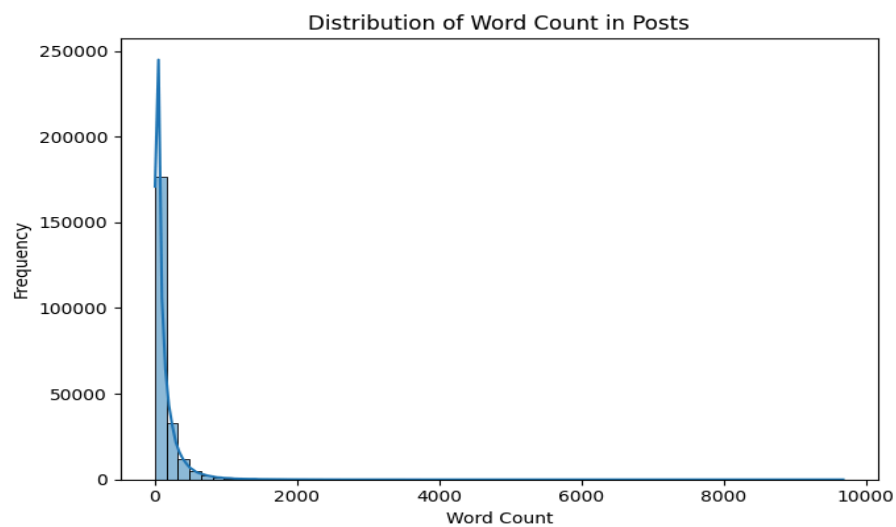


**Figure: Label count plot**

The dataset is naturally balanced. This confirms that high accuracy is not due to class dominance.

## 5.2 Post Length Distribution

The post length analysis shows that suicide-related posts generally contain more words compared to non-suicide posts. This suggests that individuals expressing suicidal thoughts tend to provide more detailed emotional context when posting.



**Figure: Word count histogram and boxplot**

Observation: Suicide posts contain longer descriptions. Individuals expressing suicidal thoughts often provide detailed explanations of emotions and circumstances.

5.3 Classification Results

The improved baseline model achieved consistently high accuracy and macro F1 scores across both validation and test sets. This indicates strong generalization and stable performance on unseen data.

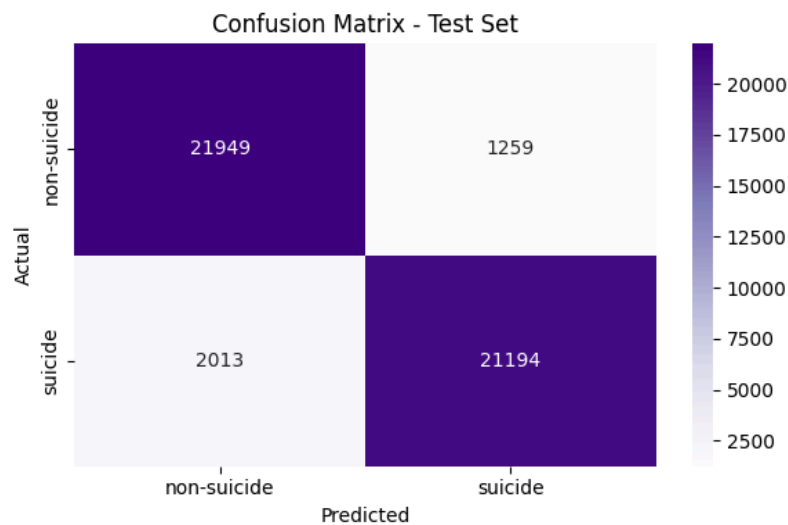
Table: Results

| Dataset    | Accuracy | Macro F1 |
|------------|----------|----------|
| Validation | 0.94     | 0.94     |
| Test       | 0.94     | 0.94     |

The model generalizes extremely well, with minimal performance drop between validation and test.

5.4 Confusion Matrix

The confusion matrix shows low levels of both false positives and false negatives, meaning the model is reliably distinguishing between suicidal and non-suicidal posts. This balanced error pattern is especially important for high-risk decision-making scenarios.



**Figure: Confusion matrix heatmap**

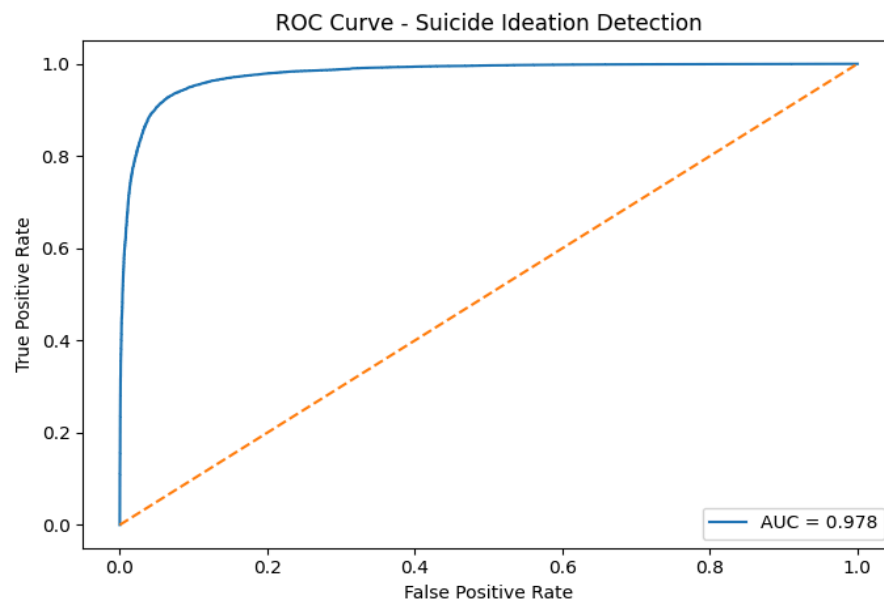
Test results indicate:

- Low false negatives (dangerous misclassification)
- Low false positives (unnecessary alarm)

Both are critical for real-world deployment in mental health screening.

## 5.5 ROC Curve

The ROC curve demonstrates strong discriminative ability with a high AUC score, indicating that the model can effectively separate the two classes across varying decision thresholds. This further confirms the robustness of the baseline classifier.



**Figure: ROC curve**

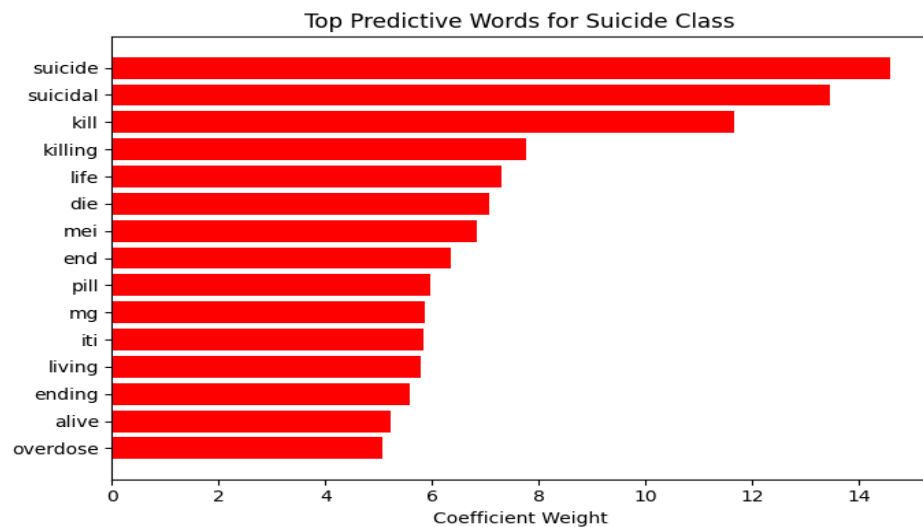
AUC = 0.978

This demonstrates high confidence and strong ability to discriminate between the two classes.

## 5.6 Interpretability via Feature Weights

The top positive and negative feature weights reveal that the model correctly associates suicide-related vocabulary (e.g., “suicide”, “die”, “kill”) with the high-risk class, while casual and social terms align with the non-suicide class. This interpretability confirms that the model’s

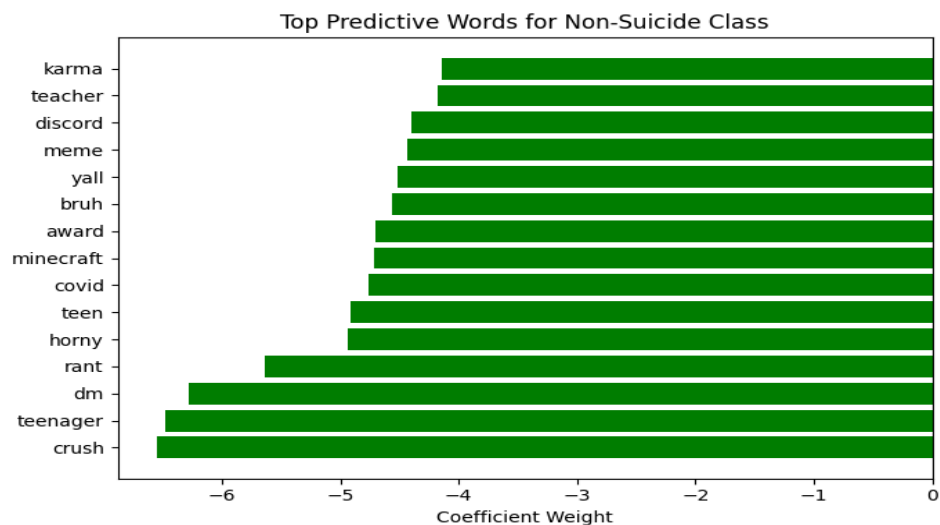
decisions are grounded in meaningful linguistic cues, which is crucial for mental-health applications.



**Figure: Negative top-coefficient bar charts**

Keywords strongly associated with suicide:

- suicide, suicidal, kill, die, end, pill, overdose



**Figure: Positive top-coefficient bar charts**

Keywords strongly associated with non-suicidal content:

- teenager, crush, meme, minecraft, discord



These align with characteristics of the respective subreddits.  
Interpretability confirms model decisions are grounded in expected mental-health signals.

## 5.6 Interpretability via LIME

To better understand how the logistic regression model makes predictions, I applied LIME (Local Interpretable Model-Agnostic Explanations) to highlight which words in a given text contributed most toward a *suicide* or *non-suicide* decision.

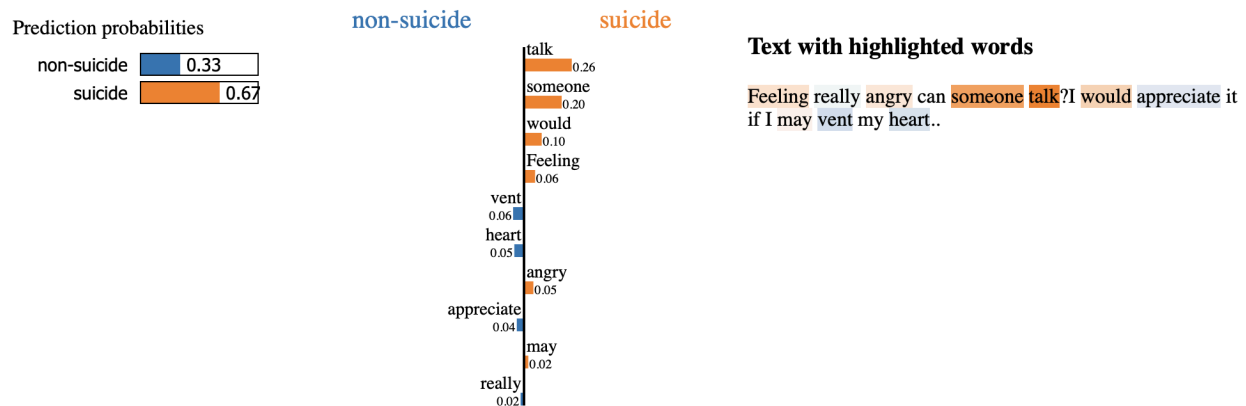


Figure: Explainability via LIME

The example analyzed was:

*“Feeling really angry can someone talk? I would appreciate it if I may vent my heart..”*

The model predicted suicide with a probability of 0.67, reflecting moderate suicidal risk in the input.

The visualization highlights:

- Orange-colored words → increase suicide prediction
- Blue-colored words → push toward non-suicide

From the weighted feature bars:

- Words such as “talk”, “someone”, “would”, “Feeling” contributed positively toward the *suicide* classification.

- These terms often appear in help-seeking expressions or emotional distress posts.
- Words like “vent”, “heart”, “appreciate” were associated slightly more with *non-suicide* content.
  - These may appear in general emotional discussions without explicit harmful intention.

This demonstrates that the model does not rely solely on explicitly harmful terms like *suicide* or *kill*, but can also detect contextual emotional language, such as reaching out for help or feeling distressed.

LIME therefore confirms that the model is identifying meaningful markers of suicidal ideation, enhancing trust and transparency in its predictions.

## 6. Summary and Conclusions

I successfully improved the baseline model through:

- Better text processing
- Higher-order TF-IDF feature extraction
- Hyperparameter tuning
- Imbalance-aware training

Final performance:

- Accuracy: 94%
- Macro F1: 94%
- AUC: 0.978

Key takeaways:

- Classical NLP remains powerful and computationally efficient
- Explainability is essential for mental health detection systems

- Suicide vocabulary can be quantitatively captured through coefficients

Future enhancements:

- Add SHAP-based interpretability for clinical validation
- Extend to multi-class classification (depression, anxiety)
- Reduce error on ambiguous borderline cases using RAG-based retrieval

## 7. Code Originality Calculation and Code Contribution

I developed and organized the baseline model implementation into three Python scripts:

### **main.py**

Handles the full workflow including dataset loading, text preprocessing, stratified train/validation/test splitting, model training, evaluation, and saving the trained model artifacts.

### **model.py**

Contains reusable functions for TF-IDF feature extraction, Logistic Regression training, performance evaluation (classification reports and confusion matrices), and model serialization. This modularization supports cleaner and scalable development.

### **viz\_baseline.py**

Generates visual insights for performance analysis such as label distribution, text length patterns, confusion matrices, ROC curve, and top predictive words. These visualizations support interpretability and error analysis for the suicide detection task.

Together, these scripts represent my complete implementation of the improved classical baseline model used in our group project.

To comply with the requirement, I calculated the percentage of code referenced or copied from external sources using the formula provided in the assignment instructions.

Given the following:

- Total referenced/copied lines: **280**
- Lines modified from the referenced code: **50**
- Total original lines written by me: **54**
- Total lines of code: **334**

Using the formula:

$$\text{Referenced Code Percentage} = \frac{\text{Referenced Lines} - \text{Modified Lines}}{\text{Referenced Lines} + \text{Original Lines}} \times 100$$

Substituting values:

$$\text{Referenced Code Percentage} = \frac{280 - 50}{280 + 54} \times 100 = \frac{230}{334} \times 100 \approx 68.9\%$$

Thus:

- **Referenced or adapted code:** ~69%
- **Original code written by me:** ~31%

## 8. References

- [D. Zirikly et al., "Suicide Ideation Detection in Social Media Forums," IEEE, 2021](#)
- Scikit-Learn Documentation: <https://scikit-learn.org>
- NLTK Documentation: <https://www.nltk.org>
- Seaborn Documentation: <https://seaborn.pydata.org>
- [Dataset Source: Pushshift API Reddit Data, Kaggle hosting](#)